

Zou_lab_control.frontend 中文手册

Jupyter / GUI 实验绘图前端

统一 array 数据契约、live session、fit、selector 与 PDF notes

2026-06-01

Contents

1	设计目标	1
1.1	这个 front-end 解决什么问题	1
1.2	两条公开路径	1
2	内部架构与数据流	2
2.1	模块分层	2
2.2	真正需要守住的状态所有权	3
2.3	一次 live update 的完整时序	3
2.4	几个关键实现为什么不能随便简化	4
2.5	源码修改时最该保护的地方	4
2.6	关键不变量	5
2.7	plot 和 run 的关系	6
2.8	RunSession 状态机和错误传播	6
2.9	Figure 生命周期和固定尺寸	6
3	安装与 notebook 使用	8
3.1	安装	8
3.2	Jupyter 初始化	8
3.3	核心数据契约	8
4	静态绘图接口	10
4.1	一维扫描	10
4.2	二维扫描	10
4.3	二维网格映射	12
4.4	plotter 初始化和更新	12
5	Live 采集接口	13
5.1	为什么是 zf.run	13
5.2	函数签名匹配	13
5.3	RunSession 内部机制	14
5.4	ArrayWatcher 和最后一点数据	14
5.5	run 参数组织	14
5.6	连续 monitor	15

6	Histogram 和阈值判别	16
6.1	双峰拟合和 threshold	16
6.2	双峰 Gaussian 拟合细节	16
7	Selector 与 DataFigure	18
7.1	交互工具	18
7.2	selector 状态模型	18
7.3	拟合与保存	19
7.4	DataFigure 拟合流程	19
8	PDF notes 接口	20
8.1	生成自定义 notes	20
8.2	notes 生成管线	20
8.3	生成 frontend manual	21
9	调试与扩展	22
9.1	常见问题定位	22
9.2	以后接 PyQt 的关键替换点	22
9.3	新增 plot 类型	23
9.4	维护原则	23

1

设计目标

1.1 这个 front-end 解决什么问题

`Zou_lab_control.frontend` 是实验控制系统和 notebook/GUI 绘图之间的薄前端层。它的核心原则是：实验逻辑只描述数据和采集，绘图层负责 figure 生命周期、固定尺寸、selector、fit、保存和 live 刷新。这样做的目的是避免把硬件控制、Jupyter 交互和 Matplotlib handle 混在一起。

数据模型保持非常小：

- `data_x` 是形状为 $(N, \text{coordinate_dim})$ 的坐标数组。
- `data_y` 是形状为 $(N, \text{channel_dim})$ 的数据数组。
- 一维扫描使用 `coordinate_dim = 1`，二维扫描使用 `coordinate_dim = 2`。
- 未完成的数据用 `NaN` 表示；front-end 可以用有限值数量推断进度。

线程边界

在 Jupyter 中，绘图刷新依赖 notebook 的事件循环；在未来 PyQt 中，Qt/Matplotlib 绘图同样必须留在主线程。阻塞式硬件采集因此不能直接跑在 UI 线程里。`zf.run` 的设计就是把采集 handle 放进 session 内部 worker，而 plot 仍然只在前端 timer 中读取 array。

1.2 两条公开路径

`zf.plot` 用于已有 array。它不会采集，只把 `data_x/data_y` 画出来，并返回稳定的 plot handle。

`zf.run` 用于阻塞式采集 handle。它创建或接收 `data_y`，内部启动 worker，前端 timer 刷新同一份数据。

Python

```
pl = zf.plot(data_x, data_y, labels=("X", "Y", "Counts"))
scan = zf.run(data_x, measure, labels=("X", "Y", "Counts"))
```

2

内部架构与数据流

2.1 模块分层

frontend 现在分成几层，每层只做一类事情。理解这个分层比记住每个参数更重要，因为以后接 PyQt 或新增实验图形时，应该尽量只替换对应层。

style.py 保存 Confocal_GUIv2 派生出来的 Matplotlib rcParams、字体和 notebook 输出样式。

canvas.py 管 figure 生命周期、固定像素数据区、cell rerun 清理、ipympl canvas 显示。

live.py 定义 plotter: [Live1D](#)、[Live2DDis](#)、[LiveLiveDis](#)、[HistogramFigure](#)，负责 artist 初始化和增量更新。

selectors.py 定义交互工具和 figure 共享状态: area、crosshair、zoom/pan、drag line。

data_figure.py 定义拟合、单位切换、保存和后处理入口。

_watcher.py 定义 frontend timer watcher，只在 UI 侧定期读取 array 并刷新 artist。

session.py 定义 [RunSession](#)，把阻塞式采集 handle 放到 session worker 里。

notes.py 定义 XeLaTeX notes/manual 生成接口。

Architecture

```
hardware/source handle
-> RunSession worker thread
    -> writes data_y under RLock
    -> updates points_done

Matplotlib/Jupyter/PyQt UI thread
-> ArrayWatcher timer
    -> snapshots data_y and points_done
    -> plot.update(...)
    -> artists, selectors, DataFigure
```

这里最关键的一点是：**采集线程永不碰 Matplotlib artist**。采集线程只写 `data_y` 和

`points_done`; UI timer 才调用 `plot.update`。这就是以后切换 PyQt 时还能保留大部分实验逻辑的原因。

2.2 真正需要守住的状态所有权

frontend 表面上只有 `plot` 和 `run`，内部其实是在管理几类寿命完全不同的状态。很多 notebook 绘图 bug 都来自这些状态被放错地方。

`data_x/data_y` 是实验数据状态。它可以被已有 array 提供，也可以由 `RunSession` 创建并逐行填充；plotter 只读取它，不拥有硬件。

`points_done/done_event/error` 是采集进度状态。它属于 `RunSession`，解决的是“array 里哪些点语义上已经完成”和“worker 是否失败”。

`fig/ax/artist` 是显示状态。它属于 plotter 和 canvas，必须留在 UI 事件循环里，不能被硬件 worker 改。

`fig._zlc_tools` 是交互 callback 的寿命状态。Matplotlib selector、drag line 和 crosshair 如果没有强引用会被垃圾回收；如果 cell rerun 不销毁旧工具，又会留下幽灵 callback。

`fig._zlc_state` 是交互工具读取数据的共享快照。selector 和 `DataFigure` 不应该直接猜 plotter 内部字段，而是通过这个轻量 state 拿最新 grid、labels、axes 和 data handle。

为什么这些状态不能合成一个大对象

一个巨大 controller 看起来能少传参数，但会让硬件 worker、Matplotlib artist、selector callback 和 experiment result 互相持有。旧 GUI 难重构的根本原因就在这里。现在每层只持有自己的状态：worker 写数据，watcher 刷新图，selector 读 figure state，DataFigure 做后处理。出了问题时，先看是哪类状态不对，再进对应文件。

2.3 一次 live update 的完整时序

维护 live 图时，最容易误判的是“图为什么没有边采边画”。下面这条时序是实际实现，不是概念图：

1. 用户调用 `zf.run(data_x, source, ...)`。这里的 `source` 可以是函数，也可以是带 `measure/read/acquire/next` 方法的对象。
2. `RunSession._prepare_arrays` 把输入标准化成 `data_x: (N,d)` 和 `data_y: (N,c)`；如果是 histogram，`run(500, source, kind="hist")` 会把 500 理解成 shot 数，并创建一列 NaN。
3. session 先调用内部 `plot(..., update="watch")`。这一步会立刻创建 figure、axes、artist、selector 和 `ArrayWatcher`，所以用户应该先看到一张空图或已有数据图。
4. `RunSession.start` 开一个 session 自己持有的 daemon worker。worker 进入 append 或 roll 循环，每次只做三件事：读 source、写 `data_y`、更新 `points_done`。
5. `ArrayWatcher` 用当前 Matplotlib canvas 的 timer 周期性执行 `refresh`。它在同一个 lock 里读取 `data_y` snapshot 和 `points_done`，再调用 `plot.update(...)`。
6. `plot.update` 不重建 figure，只进入对应 plotter 的 `update_core`：一维改 line data，二维改 image array 和 side distribution，histogram 改 bars、fit line 和 threshold 文本。
7. worker 结束后设置 `done_event`。watcher 下一次 refresh 会做最后一次完整 snapshot，然后 stop，并在需要时创建 `DataFigure`。

这条链里有两个细节很关键。第一，`RunSession` 的异常不会从 timer callback 里冒出来，而是存在 `session.error`；notebook 用户不需要等待 worker，只需要看图、必要时检查 `session.error`。第二，图如果“等采完才出现”，常见原因不是 session 没开线程，而是 source 太快，或者 notebook cell 自己在后面用 busy loop/阻塞等待占住了 kernel，让前端 timer 没机会跑。

2.4 几个关键实现为什么不能随便简化

frontend 里有几处实现看起来只是“工程细节”，但实际上是在保护 notebook 和未来 GUI 后端的边界。

RunSession._read_value 不是为了炫技式地猜签名，而是为了让硬件对象保持自然接口。一个计数器可以有 `read()`，一个扫描设备可以有 `measure(point)`，一个二维平台可以有 `measure(x, y)`，它们都不需要额外写 wrapper。session 会按 `(point, index)`、`(x, y, index)`、`(x, y)`、`(point)`、`(index)` 和 `()` 这些候选形式尝试绑定；真正采集时 worker 只看到“读一个值”这个动作。

data_y + points_done + lock 是 live plot 的最小共享状态。只用 NaN 推断进度在静态图和简单 live 中很好用，但采集线程和 UI timer 并发时，进度和 array 必须作为一个 snapshot 读取。否则二维图很容易出现最后一两个点已经写进 array，但 watcher 读到的完成点数还停在旧值，最终要点一下图才刷新。

plot.update_core 只改已有 artist。Matplotlib 在 Jupyter widget backend 下重建 axes、colorbar 或 selector callback 的成本高，而且容易留下旧 callback。因此一维只改 line，二维只改 image/side distribution，histogram 只改 bar vertices、fit line、threshold text。fit、保存和单位切换全部放进 `DataFigure`，不进入 timer 主路径。

fig._zlc_tools 与 **fig._zlc_state** 分别解决“callback 活多久”和“selector 读哪份数据”。前者保存强引用并在 cell rerun 时销毁旧工具；后者保存最新 grid、axes、labels、selector 范围和 data handle。二维 live 每次更新后都要刷新 state，否则 crosshair 或 fit 读到的是上一帧的坐标映射。

DragVLine 管 threshold 拖动，而不是让 area selector 自己判断。拖动 orange threshold 时会临时关掉 area selector，释放后恢复；这样 histogram 的阈值线、区域选择和双峰拟合文本可以共存，而不是互相抢鼠标事件。

改 live 代码时的自检顺序

先确认 `RunSession` 是否立即返回并创建空图；再确认 worker 是否只写 array 和 `points_done`；然后看 `ArrayWatcher._snapshot_and_points` 是否同锁读取；最后看对应 plotter 的 `update_core` 是否只改 artist 而不重建 figure。这个顺序基本覆盖了“只显示最终图”“最后点缺失”“selector 读旧数据”和“cell 重跑后交互异常”四类问题。

2.5 源码修改时最该保护的地方

这部分是给以后改代码的人看的。frontend 看起来只是画图，但有几处小实现是在处理 notebook 绘图的坑，改坏后现象会很怪：

- **new_figure** 只关闭同一个 cell 上一次 run 留下的 figure。不要改成全局关图，否则一个 cell 会误杀另一个 live 图。
- **fig._zlc_tools** 是强引用。Matplotlib widget callback 如果没有被保存，很容易被垃圾回收；如果 cell rerun 时没有 destroy，又会留下旧 callback。

- `fig._zlc_state` 是 selector/DataFigure 的轻量共享状态。二维图每次 `update_core` 后都要重新安装 state, 否则 crosshair 读到的 grid 会是旧的。
- `ArrayWatcher._snapshot_and_points` 必须同时读数据和进度。分开读会产生“最后两个点要点一下图才出现”这类 race。
- `HistogramFigure` 的 threshold 线是 `DragVLine` 管的; 拖动时会暂时关闭 area selector, 释放后恢复。否则拖 threshold 会顺手画出一个区域选择框。
- `DataFigure` 不应该进入 live timer 主路径。fit 可以慢, 可以失败, 可以画额外 artist; live update 必须保持成轻量的 `update_core`。

判断问题属于哪一层

如果 figure 尺寸或 colorbar 挤压不对, 看 `canvas.py`; 如果数据明明写进 array 但图不动, 看 `_watcher.py` 和 lock; 如果图动但 selector/fitting 读到旧数据, 看 `fig._zlc_state`; 如果采集没有边跑边画, 看 `session.py` 和调用 cell 是否阻塞了 notebook 事件循环。

2.6 关键不变量

这套 frontend 里有几条不变量比具体 class 名更重要。只要这些不变量保持住, Jupyter、未来 PyQt GUI、真实硬件和 virtual device 就可以共用同一套实验逻辑; 如果这些不变量被破坏, 问题通常会表现成 live 卡住、最后几个点不刷新、selector 读到旧坐标、或者 notebook 重跑后出现多个旧 callback。

1. **公开数据契约只有 `data_x/data_y`**。`data_x` 描述坐标, `data_y` 描述测量值; 静态图和 live 图的区别只是 `data_y` 是否正在从 NaN 逐步变成有限值。
2. **artist 只初始化一次**。`init_core` 创建 line、image、PolyCollection、text、drag line; `update_core` 只用 `set_data`、`set_array`、`set_verts`、`set_xdata/ydata` 改已有 artist。live 过程中不要重新 plot 或重建 axes。
3. **UI 更新只发生在 UI timer**。采集 worker 不能碰 `fig`、`ax`、Matplotlib artist 或 selector; 它只能写 array 和进度。未来 PyQt 也必须保持这个边界。
4. **`points_done` 和 `array snapshot` 要一起读**。最后几个点缺失通常不是 fit 或 imshow 的问题, 而是进度和数据在不同时间点被读到了。当前 `ArrayWatcher` 用同一个 lock 读 snapshot 和 progress, 就是为了解决这个 race。
5. **selector 状态挂在 figure 上**。`fig._zlc_tools` 防止 callback 被垃圾回收; `fig._zlc_state` 给 crosshair、zoom reset、DataFigure 读取最新 grid/axes 信息。更新图像数据后必须重新安装最新 state。
6. **DataFigure 是后处理 handle, 不是 plotter 本体**。fit、保存、单位切换、area selection 都通过它做; plotter 只负责显示和增量刷新。这样 live 刷新路径不会被拟合逻辑拖复杂。

为什么不让硬件直接拿 plot handle

让硬件代码直接调用 `plot.update_point` 看起来简单, 但会把采集时序、GUI 主线程约束和 artist 生命周期绑死。现在的设计只让硬件暴露阻塞式 `read/measure/acquire`, 由 `RunSession` 负责 worker 和停止逻辑; plotter 只看 array。这是为了让同一段实验逻辑以后能同时服务 notebook、PyQt 和 virtual-device 测试。

2.7 plot 和 run 的关系

`zf.plot` 和 `zf.run` 不是两套绘图接口。两者最后都落到同一批 `plotter class`，区别只在谁拥有采集生命周期：

- `plot(data_x, data_y)`：调用者已经有 `array`，`frontend` 只负责画。
- `run(data_x, source)`：调用者有阻塞采集 `handle`，`frontend` 创建或接管 `data_y`，并拥有 `worker`、`timer`、停止逻辑。

这也是为什么 `run` 返回的 `RunSession` 会透传很多 `plot` 属性：`session.fig`、`session.ax`、`session.data_figure` 实际上来自内部的 `session.plot`。用户面向的是一个对象，但内部仍然保留清晰分层。

2.8 RunSession 状态机和错误传播

`RunSession` 看起来像一个 `plot handle`，其实内部更像一个很小的采集状态机。这个状态机不拥有硬件语义，只拥有“什么时候开始读、读到哪里、什么时候停止、错误放在哪里”。

RunSession state

```
created
-> plot-created / watcher-started
-> worker-running
    -> write one row under lock
    -> points_done += 1
    -> repeat / roll
-> done-event set
-> watcher final refresh
-> stopped
```

`source` 抛出的异常不会跨线程直接打断 `notebook cell`，也不会从 `Matplotlib timer callback` 里弹出。`worker` 会把异常保存到 `session.error`，设置 `done event`，并让 `watcher` 有机会做最后一次 `refresh`。这样 UI 侧和硬件侧的错误通道是分开的：硬件失败看 `session.error`；图没刷新看 `watcher/timer/lock`；`fit` 失败看 `DataFigure`。

这里也故意没有把 `wait()` 做成普通用户接口。等待 `worker` 会鼓励 `notebook cell` 阻塞当前 `kernel`，而 `Jupyter/ipynb` 的 `live` 刷新正依赖这个事件循环。需要停止时调用 `session.stop()`；需要连续 `monitor` 时用 `stop_when_full=False` 让 `session` 一直滚动，实验结束再 `stop`。测试代码可以用私有同步方式验证结果，但这不是实验脚本和未来 GUI 应该依赖的接口。

2.9 Figure 生命周期和固定尺寸

`Jupyter` 里常见的问题是同一 `cell` 反复运行后旧 `selector` 还活着、`figure` 尺寸被 `colorbar` 或 `side axes` 撑变、`HiDPI canvas` 第一次显示空白。这里的解决办法集中在 `canvas.py`：

- `FigureSpec` 用 `data_px`、`margins_px` 和 `dpi` 描述逻辑像素尺寸。
- `new_figure` 读取 `notebook` 的 `cellId` 和 `message id`，只关闭同一个 `cell` 上一次运行留下的 `figure`。
- `create_axes_fixed` 用 `Matplotlib Divider` 和 `Size.Fixed` 创建固定数据区，而不是依赖 `subplots_adjust` 这种比例式布局。
- `split_axes_horizontally` 只在固定数据区内部切分 `side distribution` 和 `colorbar`，因此

主图尺寸不随附加 axes 改变。

- `display_figure` 在 `ipympl` backend 下隐藏工具栏、设置 scroll capture, 并尽量继承已知的 device pixel ratio。

为什么不用普通 subplot

普通 `plt.subplots` 的 axes 位置是 figure 比例坐标。加入 colorbar、tight layout、notebook 前端缩放之后, 真正数据区域会漂移。实验 scan 需要比较不同图之间的 spot size、ROI 和 selector 坐标, 所以固定的是数据 box, 而不是整张 figure 的外框。

3

安装与 notebook 使用

3.1 安装

推荐在仓库根目录执行 editable install。这样 notebook 不需要手动修改 `sys.path`。

Command line

```
python -m pip install -e .
```

如果当前 notebook kernel 已经 import 过旧版本包，修改源码后需要 restart kernel。Python 已加载的模块对象不会自动更新。

3.2 Jupyter 初始化

Python

```
import numpy as np
import matplotlib.pyplot as plt

import Zou_lab_control.frontend as zf

zf.use_widget_backend()
zf.enable_long_output()
zf.apply_style()
```

3.3 核心数据契约

所有 array plot 都走同一个契约：

$$\text{data_x} \in \mathbb{R}^{N \times d}, \quad \text{data_y} \in \mathbb{R}^{N \times c}.$$

其中 N 是点数， d 是坐标维度， c 是通道数。一维扫描用 $d=1$ ，二维扫描用 $d=2$ 。如果传入一维数组，内部会标准化成 $(N, 1)$ ；如果 `data_y is None`，内部会创建全 NaN 的 $(N, 1)$ array。

这个契约有三个实际好处：

- 静态图和 live 图不分两套输入；live 只是逐步把 NaN 变成有限值。
- 多通道一维线图天然支持 `data_y[:, channel]`。
- 二维 scan 不要求方形矩阵输入，实验控制只需要维护一张点表。

进度推断

如果外部没有提供 `points_done`，watcher 会用 `data_y[:, 0]` 里有限值的数量推断进度。因此当前版本默认把第一列当作主通道；如果以后需要“点有效但值可以是 NaN”的实验，需要增加显式 valid mask，而不是继续复用 NaN 语义。

4

静态绘图接口

4.1 一维扫描

`zf.plot` 会根据 `data_x` 的第二维自动选择一维或二维图。下面是一维光谱扫描和 Lorentzian fit 的例子。

Python

```
x = np.linspace(737.0, 737.2, 301).reshape(-1, 1)
y = signal.reshape(-1, 1)

ple = zf.plot(
    x,
    y,
    labels=("Wavelength (nm)", "Counts/0.1s", "Counts"),
    relim_mode="tight",
)
fit_result, popt = ple.data_figure.lorentz()
```

4.2 二维扫描

二维扫描不要求方形网格，也不要求按行顺序采集。只要 `data_x` 中每行是一个 `(x, y)` 坐标，front-end 就会把 `data_y` 填回图像网格。

Python

```
SX, SY = np.meshgrid(scan_x_axis, scan_y_axis)
data_x = np.column_stack([SX.ravel(), SY.ravel()])
data_y = counts.ravel().reshape(-1, 1)

pl_map = zf.plot(data_x, data_y, labels=("X (um)", "Y (um)", "Counts"))
pl_map.data_figure.center()
```

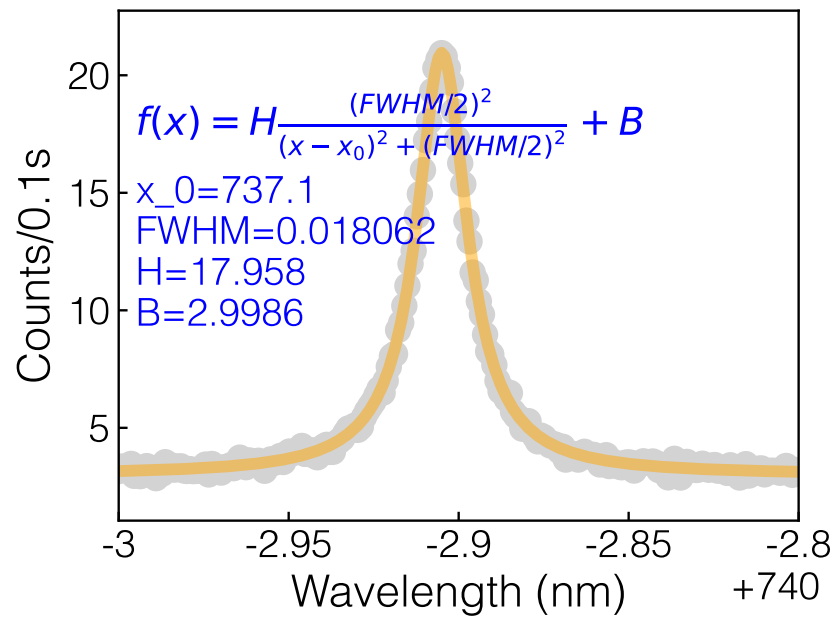


Figure 4.1: 一维扫描、固定尺寸 notebook figure、以及 `DataFigure.lorent` 拟合标注。

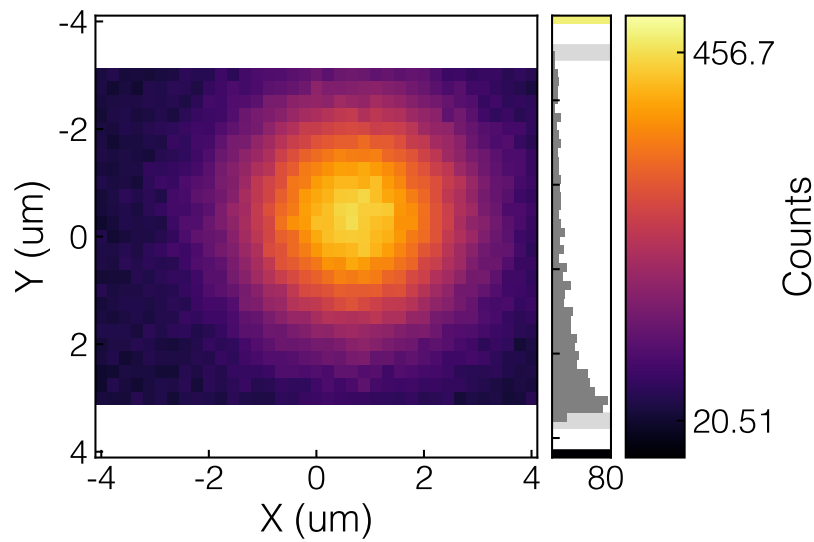


Figure 4.2: 二维 scan 图像、右侧分布、colorbar 和可拖动 color limit。

4.3 二维网格映射

Live2DDis 不把二维数据要求成 $Z.shape = (ny, nx)$ ，而是从点表恢复网格：

- `x_array = np.unique(data_x[:, 0])`, `y_array = np.unique(data_x[:, 1])`。
- `data_shape = (len(y_array), len(x_array))`，所以非方形 scan 天然支持。
- `fill_grid` 遍历每个 (x, y) , z ，用 `np.searchsorted` 找到对应像素，填入 `grid[iy, ix]`。
- `imshow` 使用由坐标间隔估计出来的 `extent`；如果 `square=True`，坐标范围会补边成正方形视野，但数据矩阵本身不需要正方形。

这种实现允许硬件按任意顺序采集点，只要坐标来自同一组离散 grid。需要注意的是，它不是三角剖分或插值绘图；如果坐标带随机 jitter，应该先在实验逻辑里 round 到真实 grid，或者以后新增一个专门的 irregular map plotter。

为什么不能简单 reshape

二维 scan 的自然数据结构是点表，不是正方形矩阵。把 `data_y` 按 `sqrt(N)` 或某个固定 `ny, nx` 直接 reshape，会把非方形 scan、蛇形 scan、随机顺序 scan 和缺点 scan 全部搞错。当前 `fill_grid` 只相信 `data_x` 里的真实坐标，NaN 仍然留成空像素；这也是 live 过程中最后点、缺点和 mask 能被统一处理的原因。

4.4 plotter 初始化和更新

每个 plotter 的生命周期都很短：

1. `show`：应用 style、创建 fixed-size axes、调用 `init_core` 初始化 artist、安装 `PlotState`、绑定 selector、display。
2. `update`：标准化新的 `data_y`、更新 `points_done`、调用 `update_core` 改已有 artist、重新安装最新 `PlotState`、draw。
3. `after_plot/to_data_figure`：在需要 fit/save 时创建 `DataFigure`。

这里刻意不在更新时重画整张 figure。live 刷新的关键是“artist 保持不变，只改数据”：一维改 line data，二维改 image array 和 side histogram vertices，histogram 改 `PolyCollection` vertices 和 threshold line 位置。

5

Live 采集接口

5.1 为什么是 `zf.run`

如果调用者自己填 array，在 Jupyter 中勉强可行；但在 PyQt 架构里，调用出的 GUI 主循环会阻塞调用栈，调用者不能再一边填 array 一边保持 UI 响应。因此 live 采集必须由一个 session/controller 接管：调用者只传普通阻塞采集函数或硬件对象。

Python

```
def measure_scan(point):
    x, y = point
    return hardware.read_at(x, y)

scan = zf.run(
    data_x,
    measure_scan,
    labels=("X", "Y", "Counts"),
    update_time=0.05,
)
```

`zf.run` 立即返回 `RunSession`，不会等待采集结束。session 暴露 `stop`、`data_x`、`data_y`、`plot`、`fig`、`ax`、`data_figure` 等常用属性。

5.2 函数签名匹配

source 可以是普通函数，也可以是有 `read`、`measure`、`acquire` 或 `next` 方法的对象。session 会按坐标维度尝试调用：

- 一维：`source(point)` 或 `source(point, index)`。
- 二维：`source(point)`，其中 point 是 `np.ndarray([x, y])`；也支持 `source(x, y)`。
- monitor/hist：可使用无参数 `source()`。

5.3 RunSession 内部机制

RunSession 做四件事：

1. `_prepare_arrays` 把输入变成统一的 `data_x/data_y`。histogram 是特殊情况：`run(200, source, kind="hist")` 表示创建 200 个 shot 的 `data_y`。
2. 构造内部 `plot(..., update="watch")`，让 `plotter` 先显示出来，并启动 `ArrayWatcher`。
3. `start` 创建一个 daemon worker thread。这个线程只调用硬件 handle，并在 `RLock` 保护下写 `data_y` 和 `points_done`。
4. `stop` 设置 stop event、尝试调用 `source.stop()`、停止 watcher，并让 session 状态进入 done。

`_read_value` 的签名适配是有意做宽的：它先找 `source` 自己是否 callable，否则查找 `measure/read/acquire/next`；然后用 `inspect.signature(...).bind(...)` 尝试 `(point, index)`、`(x, y, index)`、`(x, y)`、`(point)`、`(index)` 和 `()`。这样硬件类可以保持自然写法，不需要为了 frontend 包一层胶水函数。

5.4 ArrayWatcher 和最后一点数据

`ArrayWatcher` 是前端侧刷新器，不是采集器。它使用当前 Matplotlib backend 的 canvas timer，所以在 Jupyter widget backend 或未来 Qt backend 下，回调都发生在 UI 事件循环里。

watcher 每次 refresh 会在同一个 lock 里同时拿到 `data_y` snapshot 和 `points_done`。这个细节很重要：如果先读进度再读 array，worker 可能刚好写入下一点，导致图上缺最后一个或两个点；如果先读 array 再读进度，又可能把未完成点当成完成点。当前实现把 snapshot 和 progress 合并在一个内部调用里，done 时还会在 `copy=True` 情况下再读一次，保证最终画面完整。

Internal

```
data, points_done = watcher._snapshot_and_points()
```

Jupyter live 的限制

`zf.run` 不需要用户写线程，也没有 `wait`。但如果 `source` 是纯 Python CPU-heavy 循环并长期持有 GIL，notebook UI 仍可能不够流畅。真实硬件读取通常会阻塞在 IO 或驱动调用上，GIL 会有机会让 UI timer 执行；如果某个模拟 `source` 过快，采集可能在第一帧 timer 前就结束，这时看到的是最终图而不是慢过程。

5.5 run 参数组织

`run` 的参数分两类：

- session 参数：`mode`、`max_points`、`autostart`、`stop_when_full`、`update_time`、`copy_on_refresh`。
- plot 参数：`cmap`、`bad_color`、`thresholds`、`bins`、`relim_mode` 等。

为了以后接口更干净，plot 类型相关参数推荐放进 `plot_options` 字典；顶层 `**plot_kwargs` 仍然保留，是为了 notebook 里少写字和兼容已有调用。

这些 lifecycle 参数会故意做严格检查：`update_time` 和 `watch_interval` 必须是有限正数，`max_points` 和 histogram shot 数必须是真正的正整数，`autostart/display/stop_when_full/copy_on_refresh/copy` 必须是真正的布尔值。这里不接受 `"False"`、`True` 当作 1 或 1.0，因为 live session 一旦启动 worker，

再发现参数语义错了就很难判断问题在采集、timer 还是 plotter。早失败可以保证 bad input 在打开硬件或启动刷新之前停住。

5.6 连续 monitor

Python

```
def read_count():
    return counter.read()

monitor = zf.run(
    np.arange(200),
    read_count,
    kind="monitor",
    mode="roll",
    stop_when_full=False,
    labels=("Recent shots", "Counts/shot", "Counts"),
)

# 实验结束时停止
monitor.stop()
```

6

Histogram 和阈值判别

6.1 双峰拟合和 threshold

`kind="hist"` 面向中性原子状态判别。橙色 threshold 线可以拖动；右上角文本显示当前 threshold、根据双峰 Gaussian 拟合估计的 fidelity、实际数据按当前 threshold 分开的左右比例，以及 `fit cut`。

fit cut 是什么

`fit cut` 是双峰 Gaussian 拟合后两条 Gaussian 曲线的交点。它是拟合模型建议的切分点；真正当前使用的切分线仍然是橙色 `th`，也就是你拖动的位置。

6.2 双峰 Gaussian 拟合细节

histogram 的拟合不是直接 fit 每个 shot，而是 fit 直方图 bin center 上的 counts：

$$f(x) = A_0 e^{-(x-\mu_0)^2/(2\sigma_0^2)} + A_1 e^{-(x-\mu_1)^2/(2\sigma_1^2)}.$$

初值用当前有限数据的 median 分成左右两组，分别估计 `mu` 和 `sigma`；如果某一侧点数太少，就退回到按顺序一分为二。fit 完成后会强制让 `mu0 < mu1`，避免左/右态标签交换。

`fit cut` 的计算方式是在 `mu0` 到 `mu1` 之间取密集采样点，找到两条 Gaussian 差值最小的位置。这个值只作为模型建议，不会自动覆盖用户拖动的 threshold。

fidelity 用当前 threshold 估计：

$$F = \frac{w_0 P_0(x \leq t) + w_1 P_1(x > t)}{w_0 + w_1}, \quad w_i = |A_i \sigma_i|.$$

这里 `P0` 和 `P1` 由 Gaussian CDF 计算，`L/R` 百分比则直接由真实 shot 数据按当前 threshold 统计。因此 `fit F` 和 `L/R` 回答的是两个不同问题：前者是模型认为这条 cut 的状态判别正确率，后者是实际数据被分到左右两边的比例。

Python

```
hist = zf.plot(  
    shots,  
    kind="hist",
```

```
labels=("ROI counts", "Shots", "Population"),  
thresholds=[45],  
bins=55,  
)  
  
hist.fractions()  
hist.set_thresholds([50])
```

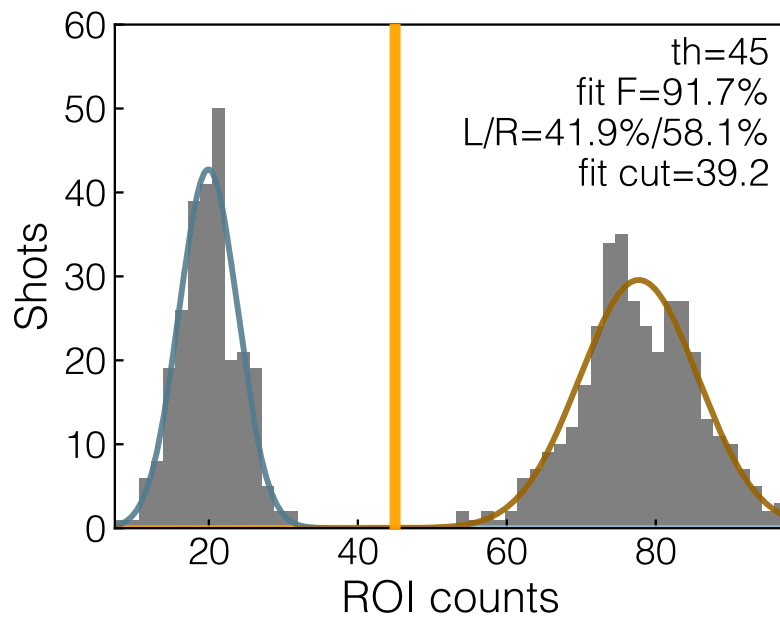


Figure 6.1: 带可拖 threshold、双峰 Gaussian 拟合和 fidelity 文本的 histogram。

7

Selector 与 DataFigure

7.1 交互工具

每个 plot handle 都保留稳定引用:

- `plot.area`: 左键矩形区域选择。
- `plot.cross`: 右键 crosshair, 连续右键两次清除。
- `plot.zoom`: 滚轮缩放, 中键拖动平移。
- `plot.drag`: 二维 color limit 或 histogram threshold 的拖动器。

拖动 histogram threshold 时, area selector 会临时关闭, 避免误触发区域选择。

7.2 selector 状态模型

Matplotlib widget 和 event callback 很容易被 Python 垃圾回收, 或者在 cell rerun 后留下旧 callback。frontend 用两个 figure 属性解决这个问题:

- `fig._zlc_tools` 保存 `InteractionBundle` 的强引用, 里面有 area、cross、zoom、drag。
- `fig._zlc_state` 保存当前 plot 的轻量状态, 例如一维 `x_array/y_array`、二维 `grid`、side distribution axes、colorbar axes 和 square extent。

`AreaSelector` 只响应左键矩形; `CrossSelector` 只响应右键, 并同时兼容 Matplotlib 的 `event.dblclick` 和两次右键时间间隔小于 0.35 s 的情况, 所以在 Jupyter/IPyMPL 里右键两次也能清掉 crosshair。二维图的 crosshair 会从 `fig._zlc_state.grid` 读最近像素的 z 值, 并在右侧分布图上加一条淡线。

`DragVLine` 用于 histogram threshold。按下 threshold 附近时, 它会设置一个 disable-area 标记, 并暂时把 area selector inactive; 释放鼠标后再恢复 area selector。这样拖 threshold 不会顺手创建一个讨厌的矩形区域。

Internal

```
fig._zlc_disable_area_once = True
```

7.3 拟合与保存

`DataFigure` 是 fit、单位切换、保存和后处理的入口。常用方法包括 `lorent`、`gaussian`、`rabi`、`decay`、`center`、`change_unit`、`change_cmap` 和 `save`。

Python

```
result, popt = ple.data_figure.lorent()
paths = ple.data_figure.save(
    "results/frontend_ple_demo",
    extra_info={"note": "tutorial"},
)
```

7.4 DataFigure 拟合流程

`DataFigure` 可以从 `plotter` 自动创建，也可以显式传入 `fig/data_x/data_y`。从 `plotter` 创建时，它会拿到同一份数据、label、selector handle 和 metadata，因此 fit 可以自然尊重当前交互状态。

fit 的内部流程是：

1. `_valid_index` 只选择 `data_y[:, 0]` 有限的点，live 未完成部分不会参与拟合。
2. `_select_fit` 根据 area selector 范围选择数据；如果没有 area，就用当前 axis limit。二维 fit 会把 area 边界对齐到最近 grid 点。
3. 每个 fit 方法构造一个或多个 `p0` 和 bounds。例如 Lorentzian 同时尝试正峰和负峰初值，Rabi 用 FFT 猜振荡频率。
4. `_fit_and_draw` 在多个初值里选择 residual 最小的结果，更新 orange fit line 或二维中心 marker。
5. `_place_text` 在几个候选角落里找和数据点重叠最少的位置放参数文本，避免每次都压在峰顶上。

一维数据点数不太多时，fit 后原来的 line 会变成淡灰 scatter，再把拟合曲线画在上面。这样视觉上更接近 Confocal_GUIv2 的风格，也更容易看 residual 和异常点。

8

PDF notes 接口

8.1 生成自定义 notes

这份 PDF 本身就是用 frontend 的 notes 接口生成的。模板来自根目录的 Stark physics notes 风格，但整理成了 XeLaTeX 版本，并加入中文、代码块、命令行块、API 引用和文件路径样式。

Python

```
body = r'''
\chapter{实验记录}
这里写中文 notes, 也可以插入 \pyapi{zf.plot} 生成的图。
'''

result = zf.render_notes_pdf(
    "notes/my_note",
    filename="my_note.tex",
    title=" 实验记录",
    subtitle="Zou lab notes",
    description=" 中文 XeLaTeX PDF",
    body=body,
)
```

8.2 notes 生成管线

`write_notes_tex` 只转义 title、subtitle、description、date; `body` 被当作 raw LaTeX 插入。这样实验 notes 可以直接写章节、公式和图片，但也意味着 body 里的下划线要放进 API/path inline 宏或 codeblock 里，或者手动转义。

模板文件作为 package data 放在 `Zou_lab_control/frontend/templates`。每次写 notes 时会先复制到输出目录，保证单独拷走 `.tex` 和 `.sty` 也能编译。编译默认用 XeLaTeX 跑两次，用来生成目录和交叉引用。

verbatim 的坑

早期版本让 codeblock 支持 LaTeX command escape, 结果如果代码块里出现 LaTeX 命令样式的字符串, TeX 会误执行它。现在 `zlcverbatim` 不启用 command chars, 代码块内容保持纯文本, 这对 tutorial 和实验记录更安全。

8.3 生成 frontend manual

Python

```
result = zf.build_frontend_manual("docs/frontend_manual")
print(result.pdf_path)
```

Command line

```
python -m pip install -e .
```


9

调试与扩展

9.1 常见问题定位

import 失败 先确认当前 Python 环境已经在仓库根目录执行过 `python -m pip install -e ..`。
如果 notebook 之前 import 过旧代码，restart kernel。

live 只有最终图 看 source 是否太快，或者当前 cell 是否用 busy loop 长时间占住 kernel。真实慢硬件应由 `zf.run` 接管；不要在 notebook 里自己 `join` worker。

2D 图缺最后点 看 `ArrayWatcher` 的 snapshot/progress 读取、`RunSession.points_done` 和 lock 是否仍然一起传递。最终 refresh 的完整性依赖这三者。

2D 网格错位 看 `data_x` 是否来自离散 grid。若坐标有浮点 jitter，`np.unique/searchsorted` 会把它当成新 grid 或放错 index。

selector 行为异常 先检查 `fig._zlc_tools` 是否存在，cell rerun 后旧 figure 是否被 `new_figure` 清掉，drag line 是否在 destroy 时断开 event callback。

hist threshold 误触发 area 看 `DragVLine` 的 area 开关和 `fig._zlc_disable_area_once`。这两个逻辑必须配合。

PDF 编译失败 先看 `frontend_manual_zh.build.log`。常见原因是没有 XeLaTeX、body raw LaTeX 里有未转义下划线，或者代码块环境被破坏。

9.2 以后接 PyQt 的关键替换点

现在 Jupyter live 使用 Matplotlib canvas timer。未来 PyQt 版本不应该改硬件层或 plotter 数据契约，主要替换三处：

- 把 `ArrayWatcher` 的 timer 调度换成 Qt 主线程 timer，例如 `QTimer`。
- 保留 `RunSession` 的 worker/read/write 边界，或者把 worker 换成 Qt thread pool，但仍然只写 array。
- 保留 plotter 的 `init_core/update_core` artist 更新方式；GUI 面板只持有 plot/session handle，不直接操作硬件采集循环。

换句话说, PyQt 版本的核心不是 “重写 frontend”, 而是把 UI 调度器从 notebook canvas timer 换成 Qt timer。只要 `data_x/data_y` 契约不动, 实验逻辑和 notebook tutorial 仍然能共用。

9.3 新增 plot 类型

新增 plot 类型时, 优先保持相同数据契约: `data_x` 描述坐标, `data_y` 描述测量值。静态绘图仍走 `zf.plot`, 阻塞采集仍走 `zf.run`。如果新增 GUI 后端, 推荐只替换 watcher/session 的 UI 调度层, 不改变实验逻辑面对的接口。

9.4 维护原则

- 不在硬件层保存 Matplotlib artist。
- 不让 plot 对象采集数据。
- 不让用户在 notebook 里手写线程或 coroutine。
- 对于 PyQt, 所有 UI 更新留在主线程; 采集 handle 放入 session worker。
- 手册和实验 notes 统一使用 `render_notes_pdf` 或 `build_frontend_manual`, 保持图和文档风格一致。